

INTRODUCTION PROJECT

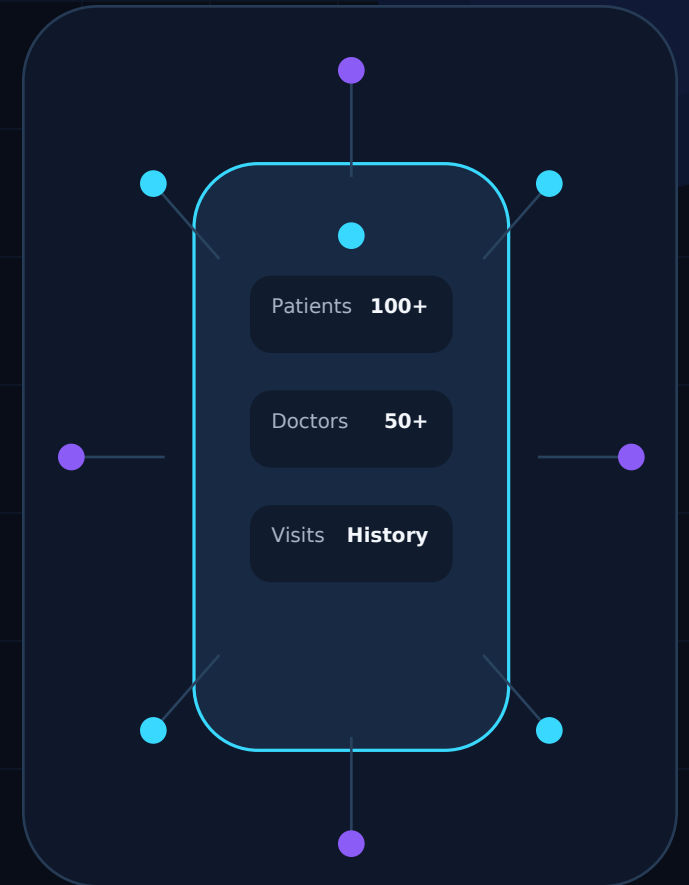
Clinic Management System

A console-based C++ system for managing patients, doctors, waiting priorities, and visit history.

Team Members

[Add team names]

C++ • OOP • Data Structures • Algorithms



GitHub: [Add repository]

What does the system do?

● CORE FEATURES

Patients

Add, view, search by ID, delete, sort

Doctors

Add and display doctor information

Waiting Room

Emergency priority + arrival order

Visits

Record diagnosis, doctor, date and fee

History

Display visits and calculate total paid

● SYSTEM FLOW

01

Register

Patient / Doctor

02

Queue

Emergency / Normal

03

Call Next

Priority-based

04

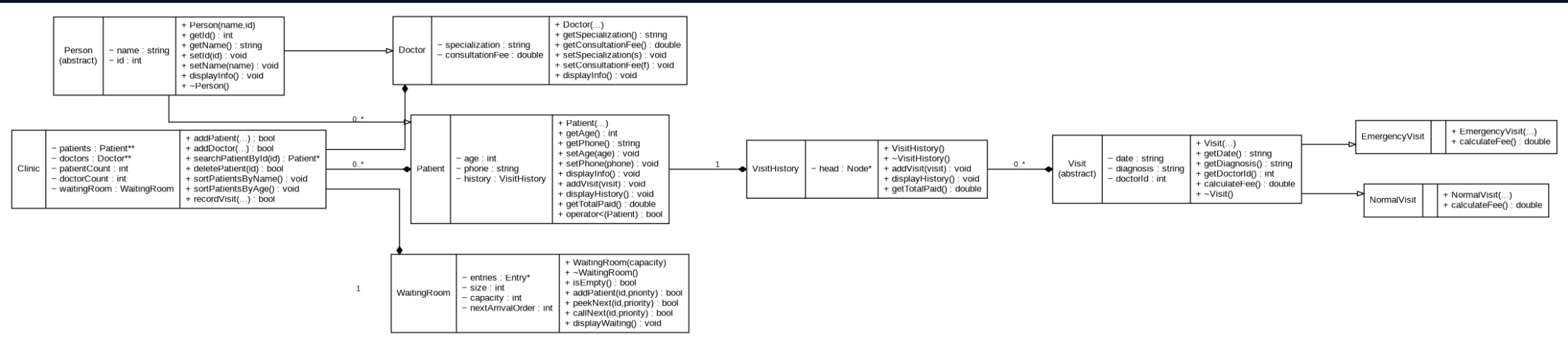
Record Visit

Doctor + Diagnosis

05

History

Fees + Total



The diagram shows the main classes, inheritance, and relationships used by the implementation.

Object-Oriented Design

ABSTRACTION

Person

Abstract base class
with pure virtual
displayInfo()

INHERITANCE

Patient / Doctor

Both inherit common
identity data from
Person

POLYMORPHISM

Visit

EmergencyVisit and
NormalVisit override
calculateFee()

ENCAPSULATION

Private Data

Access through
getters / setters

```
Visit* visit = new EmergencyVisit(...);  
double fee = visit->calculateFee(); // runtime dispatch
```

Data Structures & Algorithms

● DATA STRUCTURES

- Custom linked list for VisitHistory
Stores visit records dynamically.
- Custom priority queue for WaitingRoom
Emergency first, then arrival order.
- Dynamic pointer arrays for patients/doctors
Keeps patient IDs sorted for binary search.

● ALGORITHMS

- Binary Search Search patient by ID
- Selection Sort Sort by name / age
- Recursion Calculate total paid
- Comparison Count Measure sorting comparisons

All required structures are implemented from scratch instead of using `std::vector` / `std::queue` / `std::list` for the required DS.

Complexity at a Glance

Operation	Big O
Add Patient	$O(n)$
Search Patient	$O(\log n)$
Delete Patient	$O(n)$
Add to Waiting Room	$O(n)$
Peek Next Patient	$O(1)$
Call Next Patient	$O(n)$
Add Visit	$O(1)$
Visit History / Total	$O(v)$
Sort Patients	$O(n^2)$

KEY IDEA

Fast patient lookup

comes from keeping
patients sorted by ID

n = patients **v** = visits

Demo & Delivery

● LIVE DEMO FLOW

- 1 Add patient + doctor
- 2 Add patients to waiting room
- 3 Show Emergency before Normal
- 4 Call next patient
- 5 Record visit and fee
- 6 Display history + total

● PROJECT INFO

Repository

[\[GitHub repository link\]](#)

Team

[Member 1] [Member 2]

[Member 3] [Member 4]

Built with

C++

OOP

DSA

Thank you

Questions & live demonstration